

# Assignment 7

## Secure Network Application Development Using TCP

Report

**Prasanta Patir**

Roll No: CSB24041

ISEA Phase 3 Networking Internship — Tezpur University

*Extending the multi-client GUI TCP chat application (Assignment 6) with authentication, secure password storage, duplicate-login prevention, input validation, failed-login protection, session management and secure logging.*

## 1. Objective

This assignment enhances the GUI-based multi-client TCP chat application developed in Assignment 6 by adding practical, application-level security mechanisms. Rather than implementing advanced cryptography, the focus is on the security controls that a real client-server application needs: verifying who a user is (authentication), protecting stored credentials (SHA-256 password hashing), preventing account misuse (duplicate-login prevention and failed-login lockout), rejecting malformed input, and keeping an auditable, password-free security log of login activity.

The existing networking and GUI code from Assignment 6 was reused and extended rather than rewritten, keeping the Tkinter GUI (`client_gui.py`) separate from the socket/authentication logic (`server.py`), and the network topology was verified in Mininet before testing the application over it.

## 2. Security Features Implemented

### 2.1 User Authentication

On connection, the server immediately sends a "LOGIN" prompt to the client. The client GUI collects a username and password and returns them as a single "username:password" string. The server splits this string, and only after successful verification against the stored credential database is the client added to the active clients dictionary and allowed to send or receive chat messages.

### 2.2 Secure Password Storage (SHA-256 Hashing)

Passwords are never stored or compared in plaintext. Credentials are kept in `users.csv` as username/password-hash pairs, where the password column holds a SHA-256 hex digest produced with Python's `hashlib.sha256()`. During login, the server hashes the incoming plaintext password with the same algorithm and compares the resulting digest against the stored hash — the plaintext password itself is never written to disk or to any log file.

| username | password (SHA-256 hash)                                          |
|----------|------------------------------------------------------------------|
| James    | eba2346346bce6e941a9ff0a0c96e0e9d4282b658bd72ca2bc93d81021de08d0 |
| Patir    | 3debfe135efddea10804cafe73a1faa39dbbcd30787d1dccc0ccdcf1922970aa |
| Kuli     | 9f378328c50425014c92a99ca0799528e6b3591ea6154ddd57e626683648d10a |
| Rahul    | 687f6da20de59091e67f594827cdee268125feb3aed1c3bad77d0f6761eb198a |
| Riya     | bc97aaa7b5bde4bae9d3b658e6d4bf711b2d8bb5d7a27f17e95815efc6e0618d |

Table 1: `users.csv` — only SHA-256 password hashes are stored, never plaintext.

## 2.3 Duplicate Login Prevention

The server maintains an in-memory set, `logged_in_users`, of usernames that currently have an active session. Before verifying credentials, the server checks whether the requested username is already present in this set; if so, the connection is immediately rejected with an "ALREADY\_LOGGED\_IN" response and the attempt is recorded in the security log, without a second socket ever being registered for that user.

## 2.4 Input Validation

The server rejects malformed login requests in several ways: credential strings without a ":" separator are rejected as "INVALID"; empty usernames or empty passwords are rejected as "INVALID" and logged; and unrecognized login responses on the client side are surfaced to the user rather than silently accepted. On the chat side, TCP messages are read in bounded 1024-byte frames, which caps the size of any single message the server will process.

## 2.5 Failed Login Protection

Each username has a failed-attempt counter. A wrong password increments the counter; after five consecutive failures the account is locked for 60 seconds (`BLOCK_TIME`), during which any further login attempt — even with the correct password — is rejected with a "BLOCKED" response before the credentials are even checked. A successful login resets the counter.

## 2.6 Session Management & Secure Logging

Logout is handled through normal disconnection: when a client closes the connection (via the Disconnect button or window close), the server removes the socket from the active clients dictionary, removes the username from `logged_in_users`, broadcasts a "left the chat" notice, and appends a timestamped "Logout" entry to `security_log.txt`. Every authentication event — successful login, failed login (with source IP), duplicate-login attempt, and logout — is written to `security_log.txt` with a timestamp, and at no point is a password (plaintext or hashed) written to this file.

Note: the current implementation logs out a user based on the TCP connection closing rather than a separate inactivity timer.

# 3. System Architecture

The application follows a classic client-server model over TCP, tested on a single-switch Mininet topology with five hosts.

```
sudo mn --topo single,5 mininet> pingall mininet> nodes mininet> net
```

```
prasantapatir@prasantapatir-VirtualBox: ~/sdn7 — sudo mn --topo single,5
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4 h5
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s1) (h5, s1)
*** Configuring hosts
h1 h2 h3 h4 h5
*** Starting controller

*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5
h2 -> h1 h3 h4 h5
h3 -> h1 h2 h4 h5
h4 -> h1 h2 h3 h5
h5 -> h1 h2 h3 h4
*** Results: 0% dropped (20/20 received)
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
h4 h4-eth0:s1-eth4
h5 h5-eth0:s1-eth5
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0 s1-eth4:h4-eth0 s1-eth5:h5-eth0
mininet> nodes
available nodes are:
h1 h2 h3 h4 h5 s1
mininet>
```

Figure 1: Mininet single,5 topology — hosts h1-h5 connected via switch s1, verified with pingall (0% packet loss), nodes and net.

### 3.1 Server (server.py)

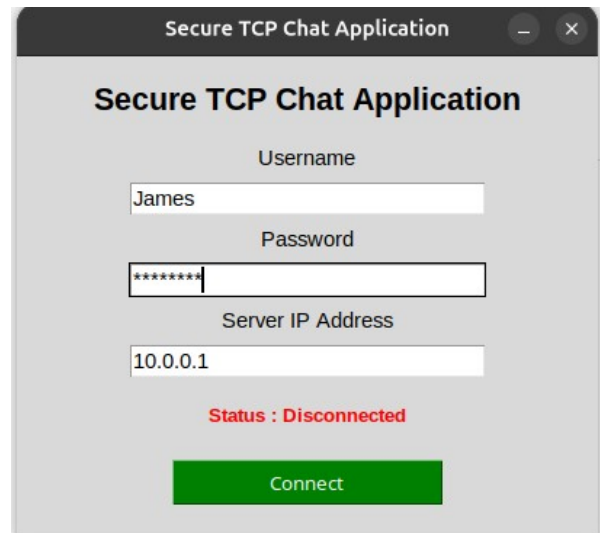
- Listens on 0.0.0.0:5000 and accepts one TCP connection per client host.
- Performs authentication (LOGIN handshake) before a client is added to the active client dictionary.
- Maintains logged\_in\_users, failed\_attempts, clients, and client\_info as shared in-memory state, protected by a threading.Lock for concurrent access.
- Spawns one daemon thread per authenticated client to handle broadcast messages, private messages (/msg <user> <text>), and the online-user list (/list).
- Persists chat history to chat\_history.csv, performance metrics to performance\_results.csv, and security events to security\_log.txt.

### 3.2 Client (client\_gui.py)

- Tkinter login window collects username, password (masked) and server IP, independent of the socket logic.
- On LOGIN\_SUCCESS the login window is hidden and a chat window opens with an online-users panel, a scrolled chat area, and a message/send/disconnect bar.
- A background thread continuously receives broadcast messages, private messages, and USERLIST updates without blocking the GUI.
- All authentication failure cases (LOGIN\_FAILED, ALREADY\_LOGGED\_IN, BLOCKED, INVALID) are shown to the user via message boxes instead of raw error text.

## 4. Implementation

### 4.1 Login Flow

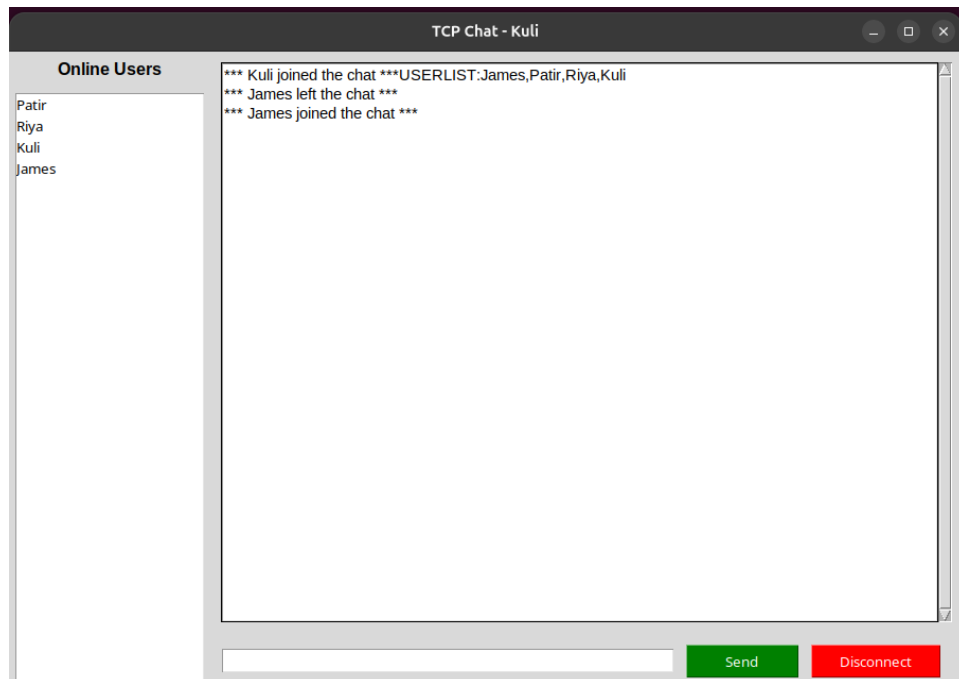


The image shows a login window titled "Secure TCP Chat Application". It contains three input fields: "Username" with the text "James", "Password" with masked characters "\*\*\*\*\*", and "Server IP Address" with the text "10.0.0.1". Below these fields, the status is displayed as "Status : Disconnected" in red text. At the bottom, there is a green "Connect" button.

Figure 2: Login window — username, masked password, and server IP fields.

The login handshake between client and server is a simple three-message exchange: the server sends the literal string LOGIN, the client replies with username:password, and the server replies with one of LOGIN\_SUCCESS, LOGIN\_FAILED, ALREADY\_LOGGED\_IN, BLOCKED, or INVALID depending on the outcome.

### 4.2 Chat Window and Multi-Client View



The image shows a chat window titled "TCP Chat - Kuli". On the left, there is a panel titled "Online Users" listing the names "Patir", "Riya", "Kuli", and "James". The main chat area displays three broadcast messages: "\*\*\* Kuli joined the chat \*\*\*USERLIST:James,Patir,Riya,Kuli", "\*\*\* James left the chat \*\*\*", and "\*\*\* James joined the chat \*\*\*". At the bottom, there is a text input field, a green "Send" button, and a red "Disconnect" button.

Figure 3: Authenticated chat window showing the online-users panel (Patir, Riya, Kuli, James) and join/leave broadcast notices.

## 4.3 Security Log Sample

An excerpt from the running application's security\_log.txt (usernames and timestamps only — no credentials):

```
2026-07-17 01:25:02.651529 : Failed login: Riya (10.0.0.5) 2026-07-17  
01:46:54.383570 : Successful login: Riya 2026-07-17 02:16:18.545454 : Duplicate  
login attempt: James 2026-07-17 02:17:57.061655 : Logout: James 2026-07-17  
02:33:55.272933 : Successful login: James
```

## 5. Testing

### 5.1 Invalid Credentials

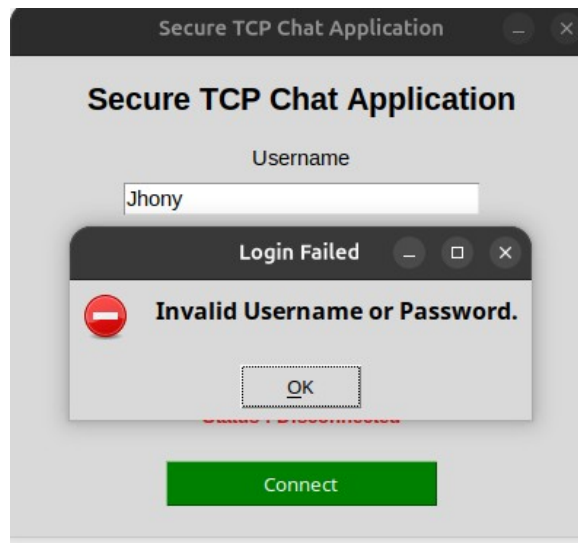


Figure 4: Unknown username ("Jhony") correctly rejected with "Invalid Username or Password."

### 5.2 Failed Login Protection (5-attempt logout)

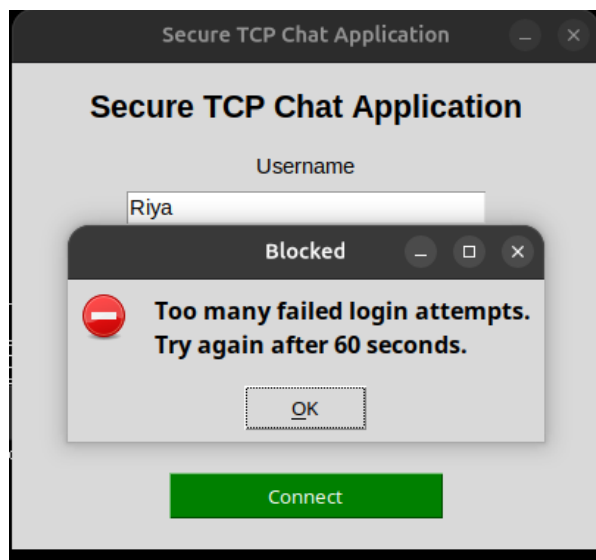


Figure 5: After five consecutive failed attempts, user "Riya" is temporarily blocked for 60 seconds.

This matches the corresponding security\_log.txt entries, which show five consecutive "Failed login: Riya" lines before a successful login is recorded once the block window elapses.

### 5.3 Duplicate Login Prevention

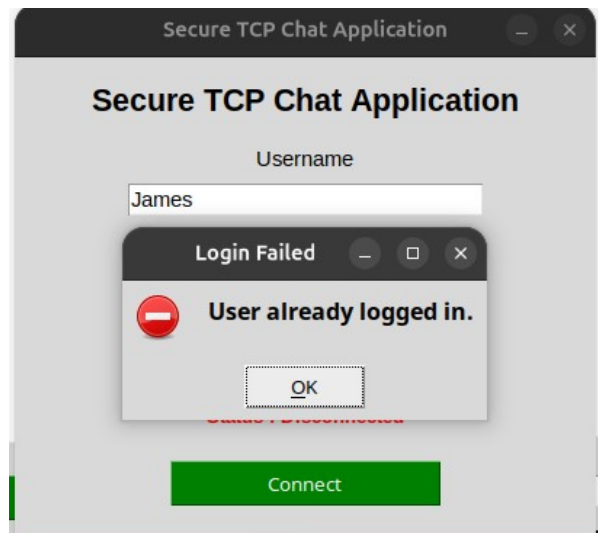


Figure 6: A second login attempt for an already-connected user ("James") is rejected client-side with "User already logged in."

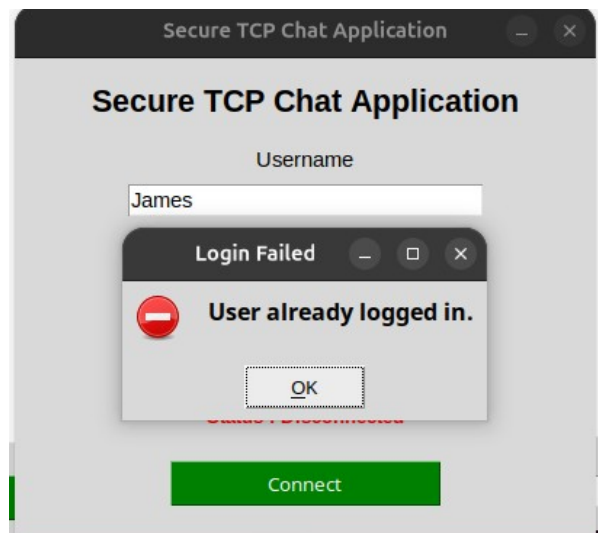


Figure 7: Duplicate-login rejection reproduced in a separate test run.

## 5.4 Online User List

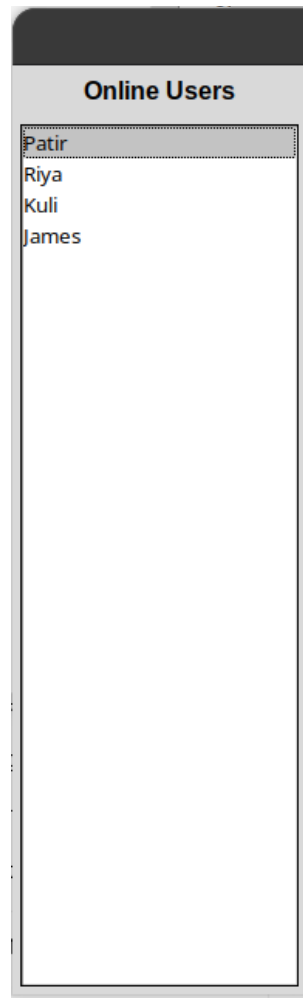


Figure 8: Online Users panel reflecting all currently authenticated clients (Patir, Riya, Kuli, James).



## 5.5 Broadcast Messaging

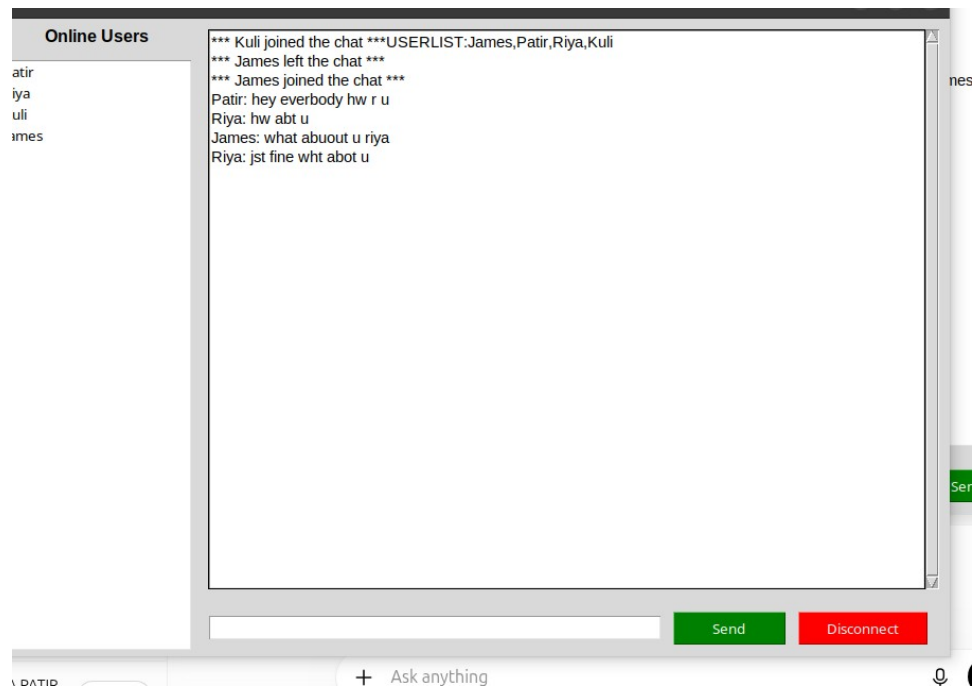


Figure 9: Broadcast chat between authenticated users, visible to all connected clients.

## 5.6 Private Messaging

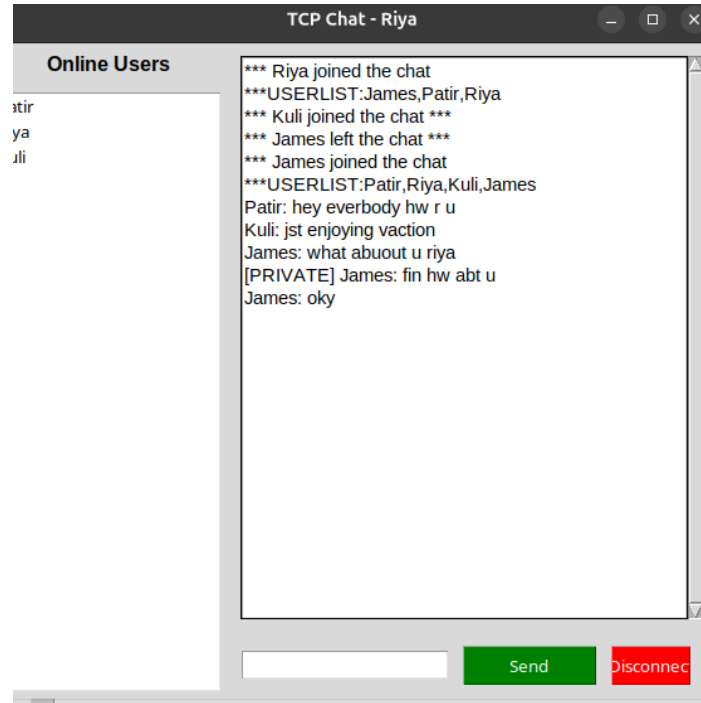


Figure 10: Private message exchange using the `/msg <username> <text>` command, delivered only to the intended recipient and logged separately in `chat_history.csv`.

## 6. Wireshark Verification

All captures were filtered with `tcp.port == 5000` to isolate traffic on the chat server's listening port.

### 6.1 Login Handshake

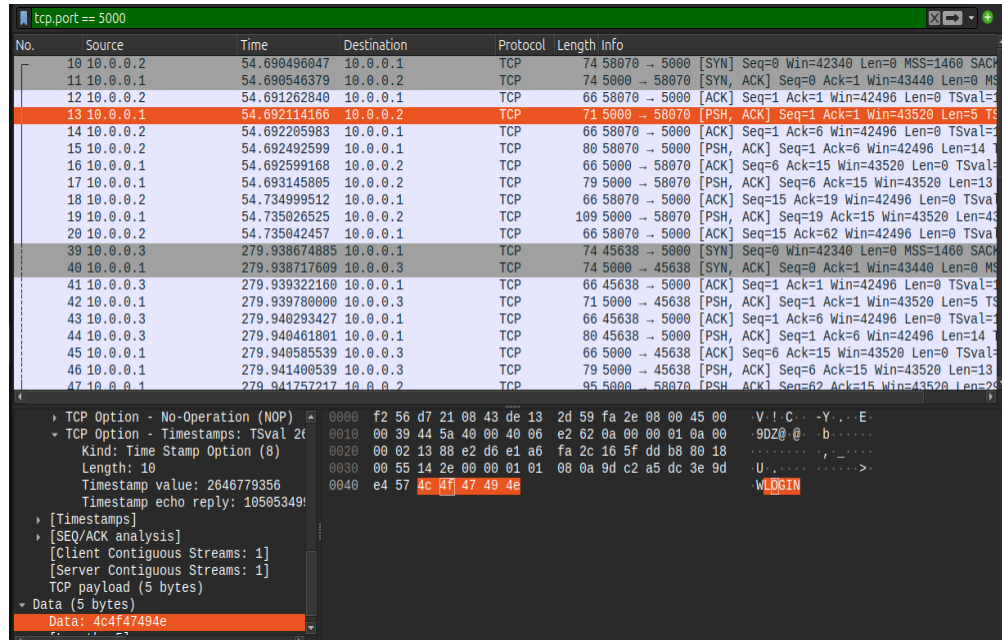


Figure 11: TCP three-way handshake (SYN, SYN-ACK, ACK) followed by the server's PSH,ACK segment carrying the 5-byte LOGIN prompt (visible as "LOGIN" in the packet bytes).

### 6.2 Successful / Authenticated Communication

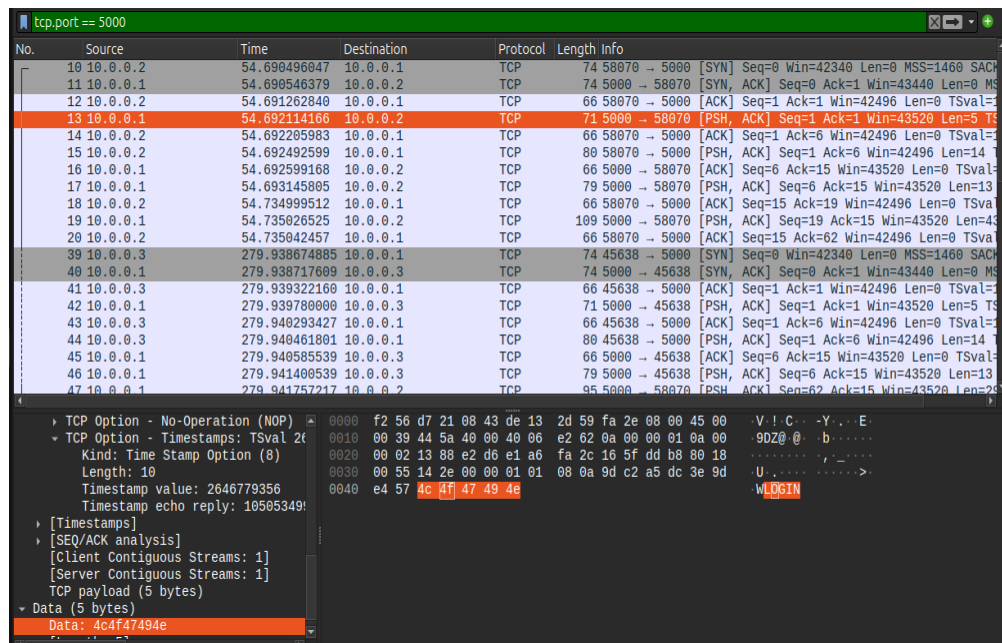


Figure 12: Client credential submission visible as application-layer payload following the LOGIN prompt (e.g. "James: h1" broadcast payload after authentication).

## 6.3 Failed Login

| No. | Source   | Time         | Destination | Protocol | Length | Info                                                       |
|-----|----------|--------------|-------------|----------|--------|------------------------------------------------------------|
| 17  | 10.0.0.2 | 58.613880012 | 10.0.0.1    | TCP      | 74     | 54518 → 5000 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 SACK...  |
| 18  | 10.0.0.1 | 58.613902690 | 10.0.0.2    | TCP      | 74     | 5000 → 54518 [SYN, ACK] Seq=0 Ack=1 Win=43440 Len=0 MSS... |
| 19  | 10.0.0.2 | 58.614036060 | 10.0.0.1    | TCP      | 66     | 54518 → 5000 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=49... |
| 20  | 10.0.0.1 | 58.614209040 | 10.0.0.2    | TCP      | 71     | 5000 → 54518 [PSH, ACK] Seq=1 Ack=1 Win=43520 Len=5 TSV... |
| 21  | 10.0.0.2 | 58.614340318 | 10.0.0.1    | TCP      | 66     | 54518 → 5000 [ACK] Seq=1 Ack=6 Win=42496 Len=0 TSval=49... |
| 22  | 10.0.0.2 | 58.615001747 | 10.0.0.1    | TCP      | 80     | 54518 → 5000 [PSH, ACK] Seq=1 Ack=6 Win=42496 Len=14 TS... |
| 23  | 10.0.0.1 | 58.615226457 | 10.0.0.2    | TCP      | 66     | 5000 → 54518 [ACK] Seq=6 Ack=15 Win=43520 Len=0 TSval=1... |
| 24  | 10.0.0.1 | 58.615801805 | 10.0.0.2    | TCP      | 79     | 5000 → 54518 [PSH, ACK] Seq=6 Ack=15 Win=43520 Len=13 T... |
| 25  | 10.0.0.2 | 58.655866257 | 10.0.0.1    | TCP      | 66     | 54518 → 5000 [ACK] Seq=15 Ack=19 Win=42496 Len=0 TSval=... |
| 26  | 10.0.0.1 | 58.655888699 | 10.0.0.2    | TCP      | 109    | 5000 → 54518 [PSH, ACK] Seq=19 Ack=15 Win=43520 Len=43...  |
| 27  | 10.0.0.2 | 58.655900406 | 10.0.0.1    | TCP      | 66     | 54518 → 5000 [ACK] Seq=15 Ack=62 Win=42496 Len=0 TSval=... |
| 32  | 10.0.0.3 | 80.334241031 | 10.0.0.1    | TCP      | 74     | 51746 → 5000 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 SACK...  |
| 33  | 10.0.0.1 | 80.334267521 | 10.0.0.3    | TCP      | 74     | 5000 → 51746 [SYN, ACK] Seq=0 Ack=1 Win=43440 Len=0 MSS... |
| 34  | 10.0.0.3 | 80.334384885 | 10.0.0.1    | TCP      | 66     | 51746 → 5000 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=66... |
| 35  | 10.0.0.1 | 80.334659342 | 10.0.0.3    | TCP      | 71     | 5000 → 51746 [PSH, ACK] Seq=1 Ack=1 Win=43520 Len=5 TSV... |
| 36  | 10.0.0.3 | 80.334675865 | 10.0.0.1    | TCP      | 66     | 51746 → 5000 [ACK] Seq=1 Ack=6 Win=42496 Len=0 TSval=66... |
| 37  | 10.0.0.3 | 80.334730407 | 10.0.0.1    | TCP      | 78     | 51746 → 5000 [PSH, ACK] Seq=1 Ack=6 Win=42496 Len=12 TS... |
| 38  | 10.0.0.1 | 80.334736648 | 10.0.0.3    | TCP      | 66     | 5000 → 51746 [ACK] Seq=6 Ack=13 Win=43520 Len=0 TSval=1... |
| 39  | 10.0.0.1 | 80.334977448 | 10.0.0.3    | TCP      | 78     | 5000 → 51746 [PSH, ACK] Seq=6 Ack=13 Win=43520 Len=12 T... |
| 40  | 10.0.0.1 | 80.337985346 | 10.0.0.3    | TCP      | 66     | 5000 → 51746 [FIN, ACK] Seq=6 Ack=13 Win=43520 Len=0 T...  |
| 41  | 10.0.0.1 | 80.342290687 | 10.0.0.3    | TCP      | 66     | [TCP Retransmission] 5000 → 51746 [FIN, ACK] Seq=18 Ac...  |
| 42  | 10.0.0.3 | 80.342234698 | 10.0.0.1    | TCP      | 78     | 51746 → 5000 [ACK] Seq=13 Ack=19 Win=42496 Len=0 TSval=... |

Frame 32: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface 0  
Ethernet II, Src: d6:07:ac:23:bf:aa (d6:07:ac:23:bf:aa), Dst: 08:00:00:00:00:00  
Internet Protocol Version 4, Src: 10.0.0.3, Dst: 10.0.0.1  
Transmission Control Protocol, Src Port: 51746, Dst Port: 5000

Figure 13: A rejected login attempt captured on tcp.port == 5000, showing the client-server exchange terminating without a subsequent authenticated data stream.

## 6.4 Broadcast and Private Message Traffic

| No. | Source   | Time          | Destination | Protocol | Length | Info                                                        |
|-----|----------|---------------|-------------|----------|--------|-------------------------------------------------------------|
| 43  | 10.0.0.3 | 279.940293427 | 10.0.0.1    | TCP      | 66     | 45638 → 5000 [ACK] Seq=1 Ack=6 Win=42496 Len=0 TSval=...    |
| 44  | 10.0.0.3 | 279.940461801 | 10.0.0.1    | TCP      | 80     | 45638 → 5000 [PSH, ACK] Seq=1 Ack=6 Win=42496 Len=14 T...   |
| 45  | 10.0.0.1 | 279.940585539 | 10.0.0.3    | TCP      | 66     | 5000 → 45638 [ACK] Seq=6 Ack=15 Win=43520 Len=0 TSval=...   |
| 46  | 10.0.0.1 | 279.941400539 | 10.0.0.3    | TCP      | 79     | 5000 → 45638 [PSH, ACK] Seq=6 Ack=15 Win=43520 Len=13 T...  |
| 47  | 10.0.0.1 | 279.941757217 | 10.0.0.2    | TCP      | 95     | 5000 → 58070 [PSH, ACK] Seq=62 Ack=15 Win=43520 Len=29...   |
| 48  | 10.0.0.2 | 279.942933466 | 10.0.0.1    | TCP      | 66     | 58070 → 5000 [ACK] Seq=15 Ack=91 Win=42496 Len=0 TSval=...  |
| 49  | 10.0.0.1 | 279.945006249 | 10.0.0.2    | TCP      | 66     | 5000 → 58070 [PSH, ACK] Seq=91 Ack=15 Win=43520 Len=20...   |
| 50  | 10.0.0.2 | 279.945936447 | 10.0.0.1    | TCP      | 66     | 58070 → 5000 [ACK] Seq=15 Ack=111 Win=42496 Len=0 TSval=... |
| 51  | 10.0.0.3 | 279.981922869 | 10.0.0.1    | TCP      | 66     | 45638 → 5000 [ACK] Seq=15 Ack=19 Win=42496 Len=0 TSval=...  |
| 52  | 10.0.0.1 | 279.981943703 | 10.0.0.3    | TCP      | 115    | 5000 → 45638 [PSH, ACK] Seq=19 Ack=15 Win=43520 Len=49...   |
| 53  | 10.0.0.3 | 279.981955229 | 10.0.0.1    | TCP      | 66     | 45638 → 5000 [ACK] Seq=15 Ack=68 Win=42496 Len=0 TSval=...  |
| 60  | 10.0.0.2 | 481.310771672 | 10.0.0.1    | TCP      | 66     | 58070 → 5000 [PSH, ACK] Seq=15 Ack=111 Win=42496 Len=2...   |
| 61  | 10.0.0.1 | 481.324297497 | 10.0.0.3    | TCP      | 79     | 5000 → 45638 [PSH, ACK] Seq=68 Ack=15 Win=43520 Len=13 T... |
| 62  | 10.0.0.3 | 481.324297497 | 10.0.0.1    | TCP      | 66     | 45638 → 5000 [ACK] Seq=13 Ack=77 Win=42496 Len=0 TSval=...  |
| 63  | 10.0.0.1 | 481.353036366 | 10.0.0.2    | TCP      | 66     | 5000 → 58070 [ACK] Seq=111 Ack=17 Win=43520 Len=0 TSval=... |
| 64  | 10.0.0.3 | 485.276399292 | 10.0.0.1    | TCP      | 69     | 45638 → 5000 [PSH, ACK] Seq=15 Ack=77 Win=42496 Len=3...    |
| 65  | 10.0.0.1 | 485.278311004 | 10.0.0.2    | TCP      | 76     | 5000 → 58070 [PSH, ACK] Seq=111 Ack=17 Win=43520 Len=1...   |
| 66  | 10.0.0.2 | 485.278413735 | 10.0.0.1    | TCP      | 66     | 58070 → 5000 [ACK] Seq=17 Ack=121 Win=42496 Len=0 TSval=... |
| 67  | 10.0.0.1 | 485.332245474 | 10.0.0.3    | TCP      | 66     | 5000 → 45638 [ACK] Seq=77 Ack=18 Win=43520 Len=0 TSval=...  |

TCP Option - No-Operation (NOP) 0000 ea 46 f6 76 fc 29 de 13 2d 59 fa 2e 08 00 45 00 F.v) -Y...E  
TCP Option - Timestamps: TSval 68 0010 00 3d 68 e3 40 00 40 06 bd d4 0a 00 00 01 0a 00 =h@...  
Kind: Time Stamp Option (8) 0020 00 03 13 88 b2 46 0b e4 18 09 6b 58 9c 42 80 18 ....F...KX B...  
Length: 10 0030 00 55 14 33 00 00 01 01 08 0a 29 12 97 4e 09 82 U.3...N...  
Timestamp value: 689084238 0040 3b 05 4a 01 6d 65 73 3a 20 68 69 ; James: hi  
Timestamp echo reply: 15952973:  
[Timestamps]  
[SEQ/ACK analysis]  
[Client Contiguous Streams: 1]  
[Server Contiguous Streams: 1]  
TCP payload (9 bytes)  
Data (9 bytes)  
Data: 4a016d65733a206869

Figure 14: Broadcast message segment — payload "/msg James hi w james" visible in the TCP data, confirming private-message routing at the packet level.

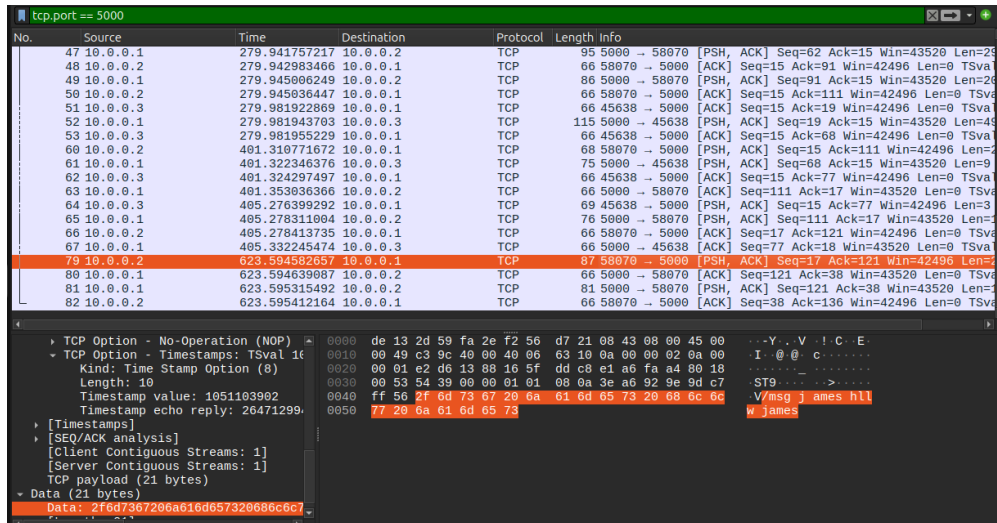


Figure 15: Server-to-client PSH,ACK segment carrying the private-message payload ("\*\*\* James left the chat \*\*\*" / related session traffic) on port 5000.

## 6.5 Logout / Disconnection

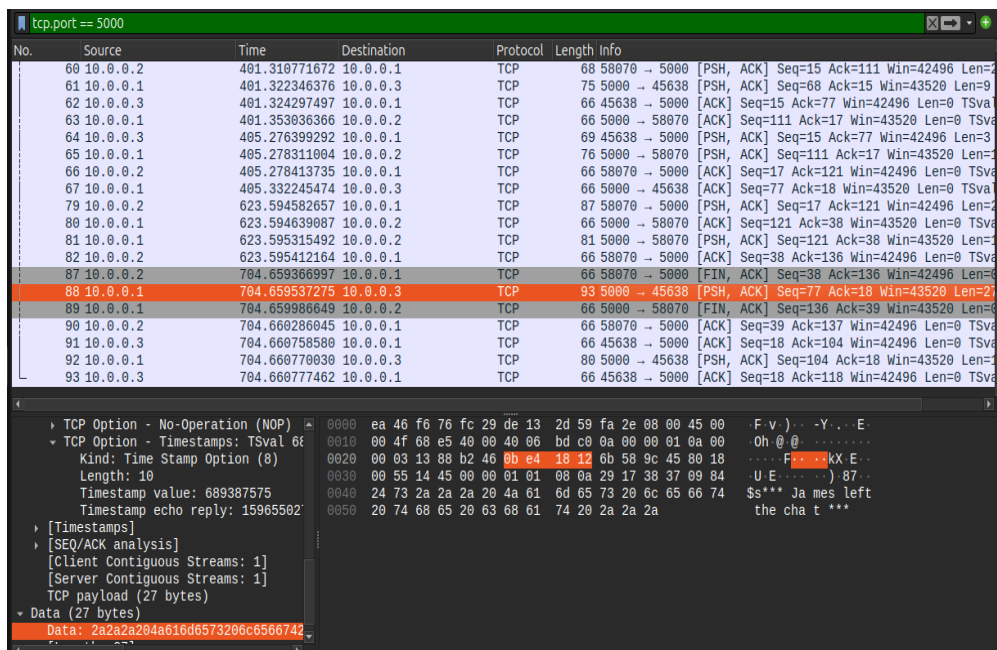


Figure 16: Client-initiated FIN,ACK segments on tcp.port == 5000 marking session teardown, corresponding to a "Logout" entry in security\_log.txt.

Because the application protocol is plaintext TCP (no TLS), Wireshark can read the LOGIN prompt and message payloads directly, which is useful for verifying protocol behaviour during development and grading, but confirms that credentials and chat content are sent unencrypted on the wire.

## 7. Conclusion

The application successfully extends the Assignment 6 multi-client TCP chat system with practical, testable security controls: credential-based authentication, SHA-256 password hashing (with no plaintext password ever stored or logged), duplicate-login prevention using an in-memory active-user set, basic input validation on the login handshake, a five-attempt/60-second lockout for failed logins, and a timestamped, password-free security log covering logins, failed logins, duplicate attempts, and logouts. All six behaviours were verified both functionally (GUI screenshots) and at the packet level (Wireshark, `tcp.port == 5000`) on a Mininet single,5 topology.